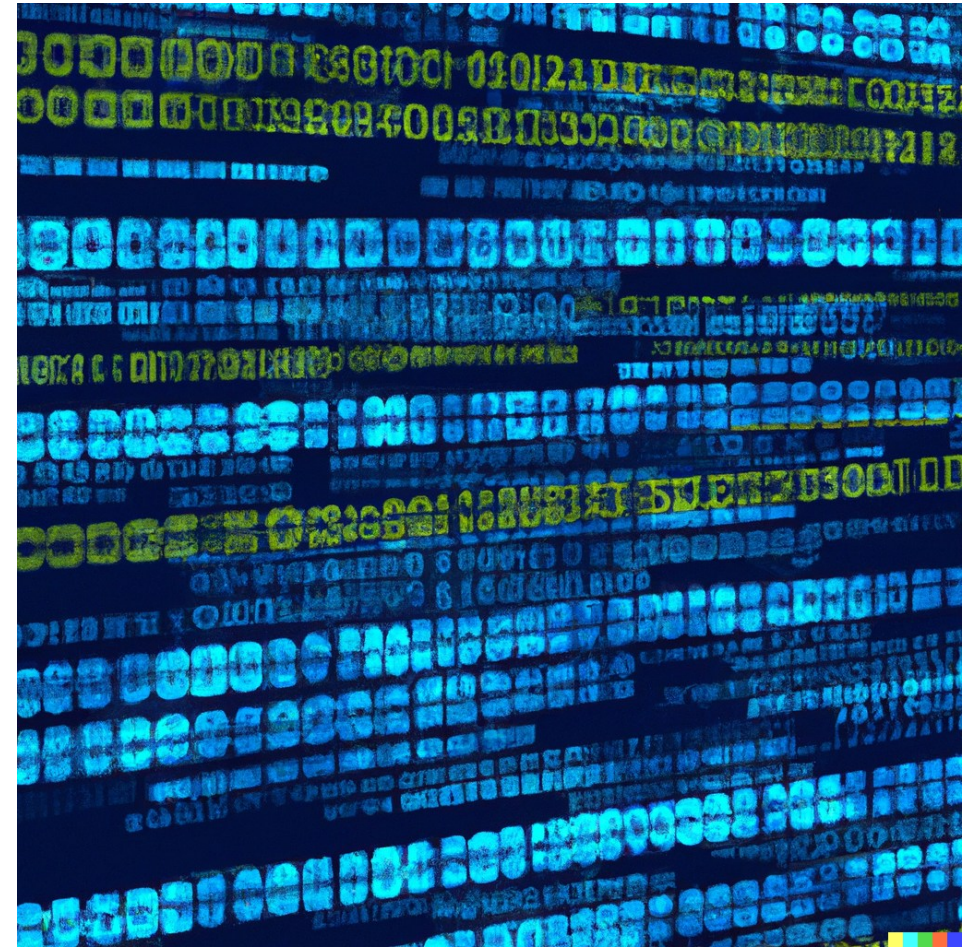


Data Ingestion and Storage

Moving, Storing and Processing data in AWS

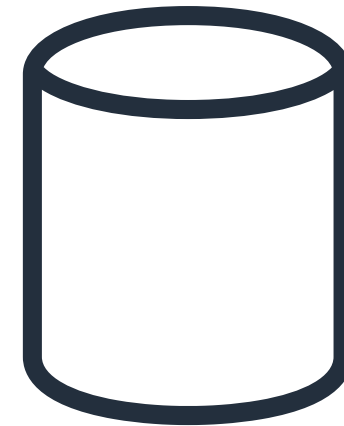
Types of Data

- Structured
- Unstructured
- Semi-Structured



Structured Data

- Definition: Data that is organized in a defined manner or schema, typically found in relational databases.
- Characteristics:
 - Easily queryable
 - Organized in rows and columns
 - Has a consistent structure
- Examples:
 - Database tables
 - CSV files with consistent columns
 - Excel spreadsheets



Unstructured Data

- Definition: Data that doesn't have a predefined structure or schema.
- Characteristics:
 - Not easily queryable without preprocessing
 - May come in various formats
- Examples:
 - Text files without a fixed format
 - Videos and audio files
 - Images
 - Emails and word processing documents



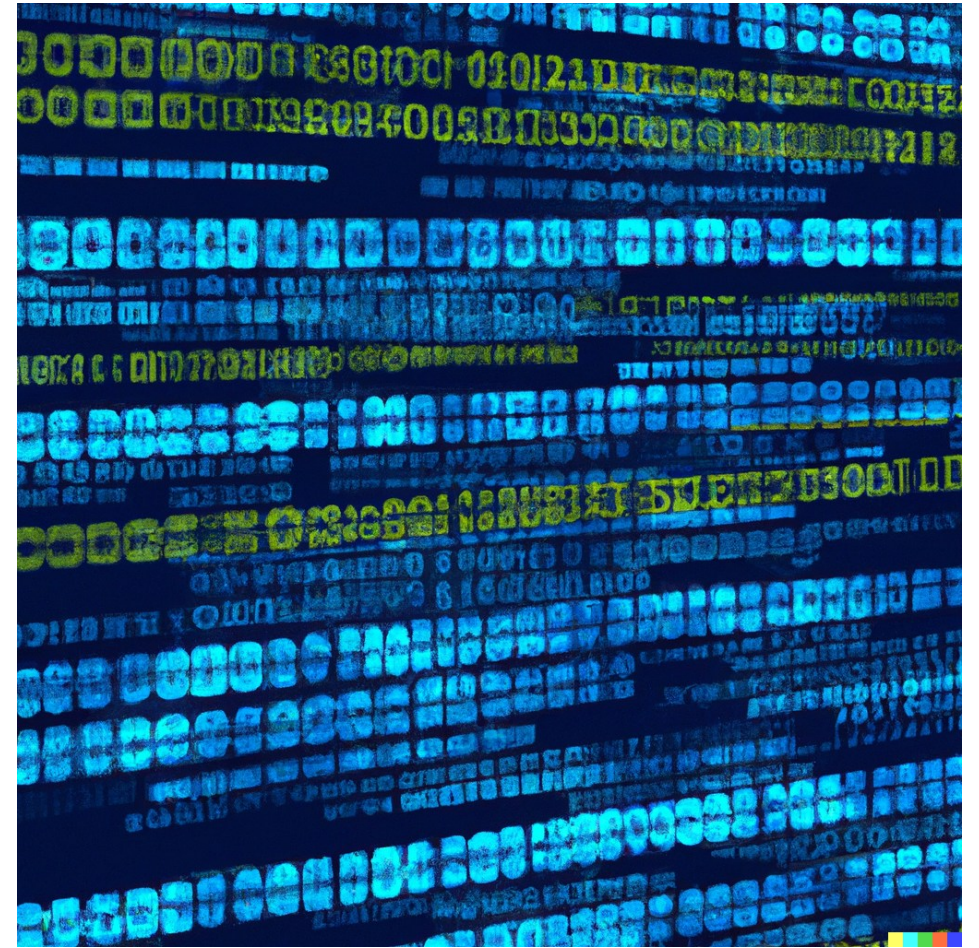
Semi-Structured Data

- Definition: Data that is not as organized as structured data but has some level of structure in the form of tags, hierarchies, or other patterns.
- Characteristics:
 - Elements might be tagged or categorized in some way
 - More flexible than structured data but not as chaotic as unstructured data
- Examples:
 - XML and JSON files
 - Email headers (which have a mix of structured fields like date, subject, etc., and unstructured data in the body)
 - Log files with varied formats



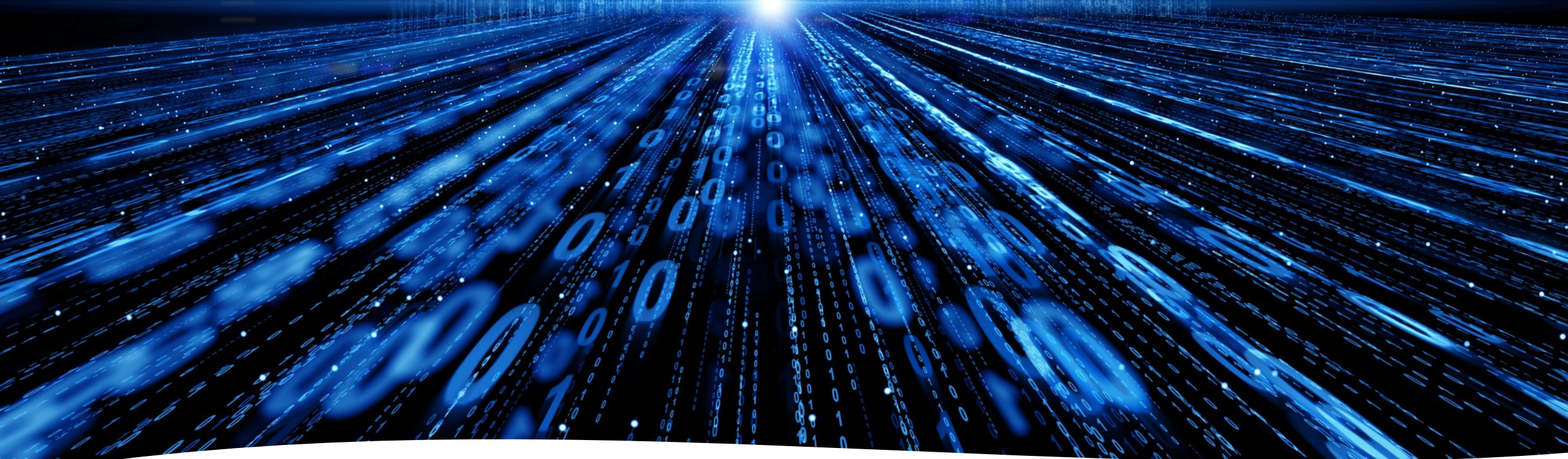
Properties of Data

- Volume
- Velocity
- Variety



Volume

- Definition: Refers to the amount or size of data that organizations are dealing with at any given time.
- Characteristics:
 - May range from gigabytes to petabytes or even more
 - Challenges in storing, processing, and analyzing high volumes of data
- Examples:
 - A popular social media platform processing terabytes of data daily from user posts, images, and videos.
 - Retailers collecting years' worth of transaction data, amounting to several petabytes.



Velocity

- Definition: Refers to the speed at which new data is generated, collected, and processed.
- Characteristics:
 - High velocity requires real-time or near-real-time processing capabilities
 - Rapid ingestion and processing can be critical for certain applications
- Examples:
 - Sensor data from IoT devices streaming readings every millisecond.
 - High-frequency trading systems where milliseconds can make a difference in decision-making.



Variety

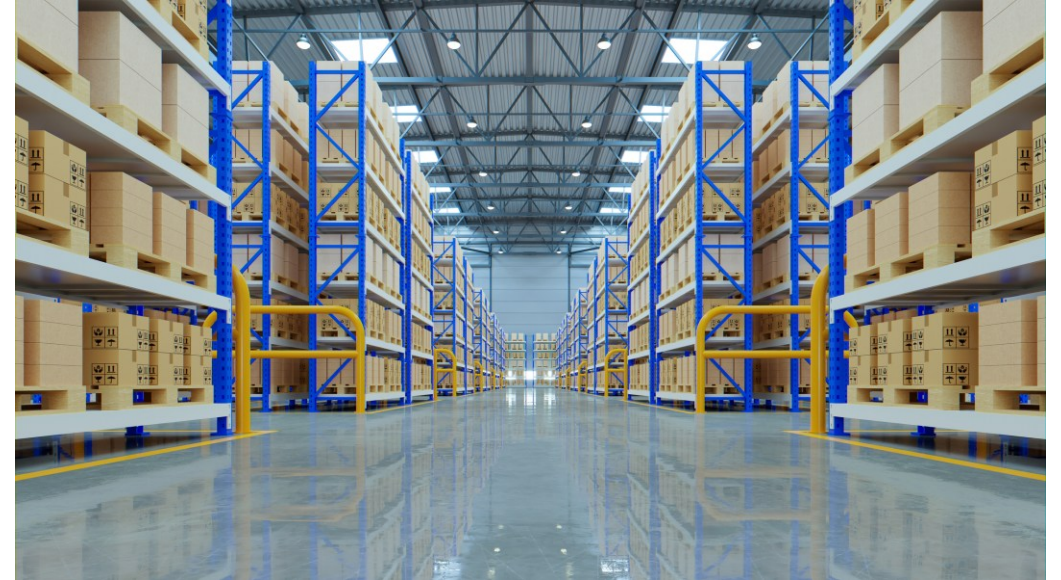
- Definition: Refers to the different types, structures, and sources of data.
- Characteristics:
 - Data can be structured, semi-structured, or unstructured
 - Data can come from multiple sources and in various formats
- Examples:
 - A business analyzing data from relational databases (structured), emails (unstructured), and JSON logs (semi-structured).
 - Healthcare systems collecting data from electronic medical records, wearable health devices, and patient feedback forms.



Data Warehouses vs. Data Lakes

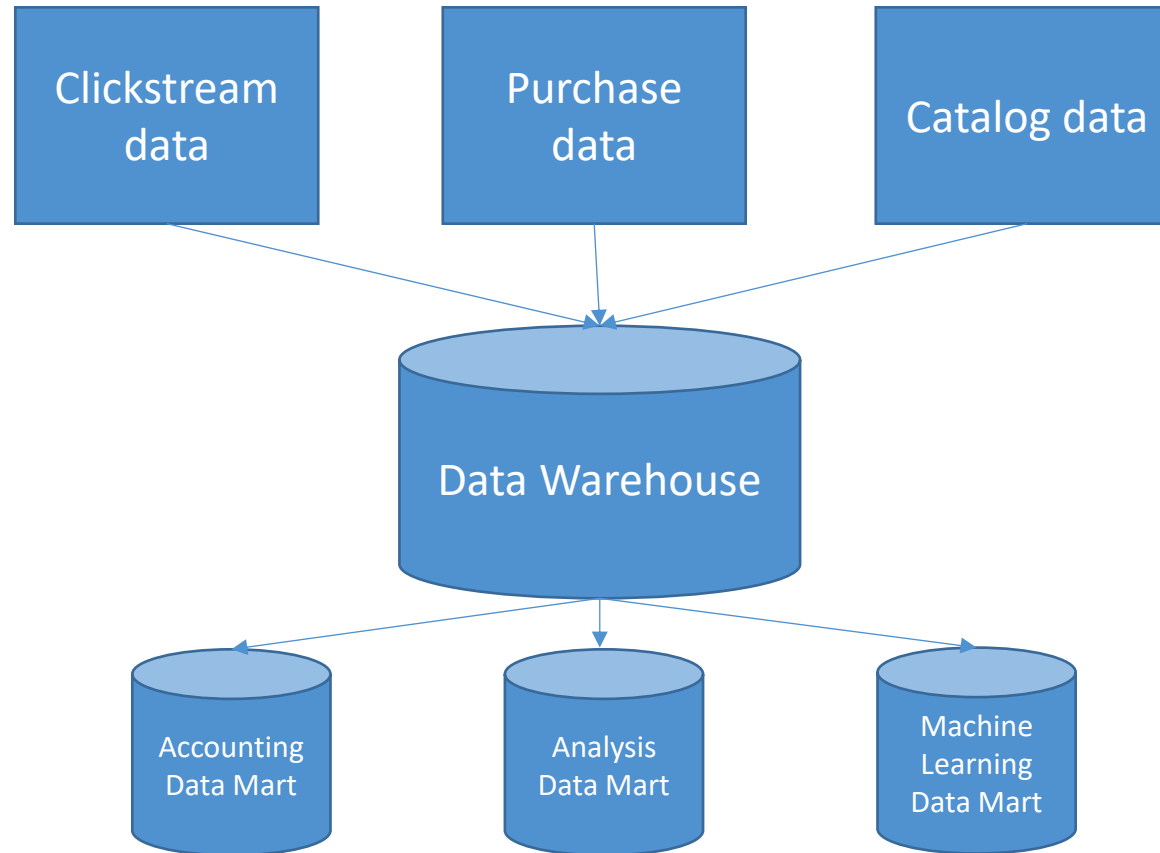
Data Warehouse

- Definition: A centralized repository optimized for analysis where data from different sources is stored in a structured format.
- Characteristics:
 - Designed for complex queries and analysis
 - Data is cleaned, transformed, and loaded (ETL process)
 - Typically uses a star or snowflake schema
 - Optimized for read-heavy operations
- Examples:
 - Amazon Redshift
 - Google BigQuery
 - Microsoft Azure SQL Data Warehouse



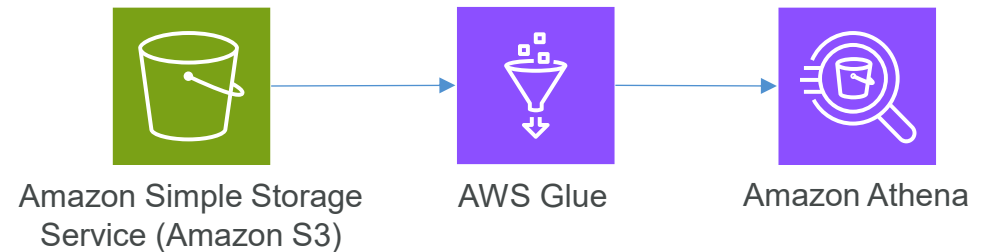
Amazon Redshift

Data Warehouse example



Data Lake

- Definition: A storage repository that holds vast amounts of raw data in its native format, including structured, semi-structured, and unstructured data.
- Characteristics:
 - Can store large volumes of raw data without predefined schema
 - Data is loaded as-is, no need for preprocessing
 - Supports batch, real-time, and stream processing
 - Can be queried for data transformation or exploration purposes
- Examples:
 - Amazon Simple Storage Service (S3) when used as a data lake
 - Azure Data Lake Storage
 - Hadoop Distributed File System (HDFS)



Comparing the two

- **Schema:**
 - Data Warehouse: Schema-on-write (predefined schema before writing data)
 - Extract – Transform – Load (ETL)
 - Data Lake: Schema-on-read (schema is defined at the time of reading data)
 - Extract – Load – Transform (ELT)
- **Data Types:**
 - Data Warehouse: Primarily structured data
 - Data Lake: Both structured and unstructured data
- **Agility:**
 - Data Warehouse: Less agile due to predefined schema
 - Data Lake: More agile as it accepts raw data without a predefined structure
- **Processing:**
 - Data Warehouse: ETL (Extract, Transform, Load)
 - Data Lake: ELT (Extract, Load, Transform) or just Load for storage purposes
- **Cost:**
 - Data Warehouse: Typically more expensive because of optimizations for complex queries
 - Data Lake: Cost-effective storage solutions, but costs can rise when processing large amounts of data

Choosing a Warehouse vs. a Lake

- **Use a Data Warehouse when:**
 - You have structured data sources and require fast and complex queries.
 - Data integration from different sources is essential.
 - Business intelligence and analytics are the primary use cases.
- **Use a Data Lake when:**
 - You have a mix of structured, semi-structured, or unstructured data.
 - You need a scalable and cost-effective solution to store massive amounts of data.
 - Future needs for data are uncertain, and you want flexibility in storage and processing.
 - Advanced analytics, machine learning, or data discovery are key goals.
- Often, organizations use a combination of both, ingesting raw data into a data lake and then processing and moving refined data into a data warehouse for analysis.



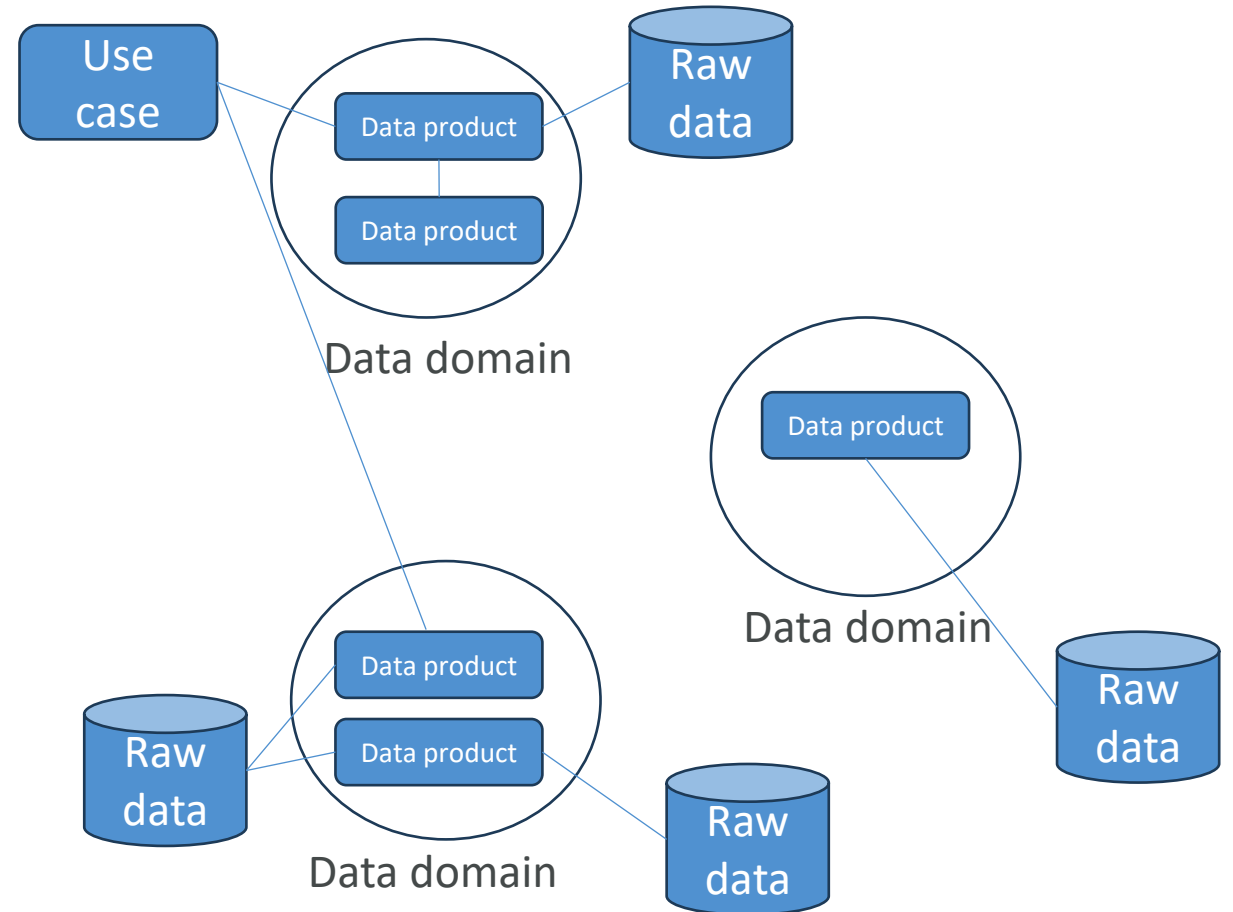
Data Lakehouse

- Definition: A hybrid data architecture that combines the best features of data lakes and data warehouses, aiming to provide the performance, reliability, and capabilities of a data warehouse while maintaining the flexibility, scale, and low-cost storage of data lakes.
- Characteristics:
 - Supports both structured and unstructured data.
 - Allows for schema-on-write and schema-on-read.
 - Provides capabilities for both detailed analytics and machine learning tasks.
 - Typically built on top of cloud or distributed architectures.
 - Benefits from technologies like Delta Lake, which bring ACID transactions to big data.
- Examples:
 - **AWS Lake Formation (with S3 and Redshift Spectrum)**
 - **Delta Lake:** An open-source storage layer that brings ACID transactions to Apache Spark and big data workloads.
 - **Databricks Lakehouse Platform:** A unified platform that combines the capabilities of data lakes and data warehouses.
 - **Azure Synapse Analytics:** Microsoft's analytics service that brings together big data and data warehousing.



Data Mesh

- Coined in 2019; it's more about governance and organization
- Individual teams own “data products” within a given domain
- These data products serve various “use cases” around the organization
- “Domain-based data management”
- Federated governance with central standards
- Self-service tooling & infrastructure
- Data lakes, warehouses, etc. may be part of it
 - But a “data mesh” is more about the “data management paradigm” and not the specific technologies or architectures.



ETL Pipelines

- **Definition:** ETL stands for Extract, Transform, Load. It's a process used to move data from source systems into a data warehouse.
- **Extract:**
 - Retrieve raw data from source systems, which can be databases, CRMs, flat files, APIs, or other data repositories.
 - Ensure data integrity during the extraction phase.
 - Can be done in real-time or in batches, depending on requirements.

ETL Pipelines: Transform

- Convert the extracted data into a format suitable for the target data warehouse.
- Can involve various operations such as:
 - Data cleansing (e.g., removing duplicates, fixing errors)
 - Data enrichment (e.g., adding additional data from other sources)
 - Format changes (e.g., date formatting, string manipulation)
 - Aggregations or computations (e.g., calculating totals or averages)
 - Encoding or decoding data
 - Handling missing values

ETL Pipelines: Load

- Move the transformed data into the target data warehouse or another data repository.
- Can be done in batches (all at once) or in a streaming manner (as data becomes available).
- Ensure that data maintains its integrity during the loading phase.



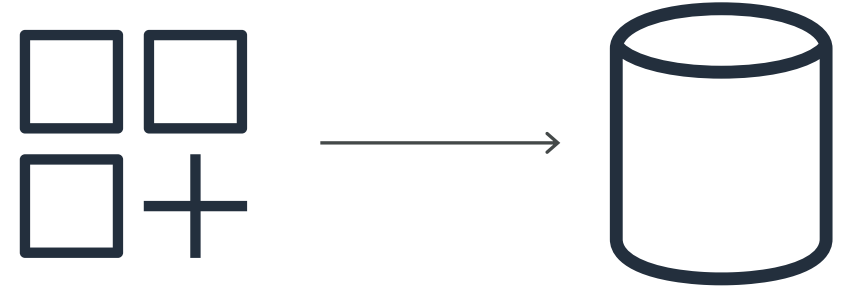
Managing ETL Pipelines

- This process must be automated in some reliable way
- AWS Glue
- Orchestration services
 - EventBridge
 - Amazon Managed Workflows for Apache Airflow [Amazon MWAA]
 - AWS Step Functions
 - Lambda
 - Glue Workflows
- We'll get into specific architectures later.



Data Sources

- JDBC
 - Java Database Connectivity
 - Platform-independent
 - Language-dependent
- ODBC
 - Open Database Connectivity
 - Platform-dependent (thx to drivers)
 - Language-independent
- Raw logs
- API's
- Streams



CSV (Comma-Separated Values)

- **Description:** Text-based format that represents data in a tabular form where each line corresponds to a row and values within a row are separated by commas.
- **When to Use:**
 - For small to medium datasets.
 - For data interchange between systems with different technologies.
 - For human-readable and editable data storage.
 - Importing/Exporting data from databases or spreadsheets.
- **Systems:** Databases (SQL-based), Excel, Pandas in Python, R, many ETL tools.

```
InvoiceNo,StockCode,Description,Quantity,InvoiceDate,UnitPrice,CustomerID,Country
536365,85123A,WHITE HANGING HEART T-LIGHT HOLDER,6,12/1/2010,8:26,2.55,17850,United Kingdom
536365,71053,WHITE METAL LANTERN,6,12/1/2010,8:26,3.39,17850,United Kingdom
536365,84406B,CREAM CUPID HEARTS COAT HANGER,8,12/1/2010,8:26,2.75,17850,United Kingdom
536365,84029G,KNITTED UNION FLAG HOT WATER BOTTLE,6,12/1/2010,8:26,3.39,17850,United Kingdom
536365,84029E,RED WOOLLY HOTTIE WHITE HEART.,6,12/1/2010,8:26,3.39,17850,United Kingdom
536365,22752,SET 7 BABUSHKA NESTING BOXES,2,12/1/2010,8:26,7.65,17850,United Kingdom
536365,21730,GLASS STAR FROSTED T-LIGHT HOLDER,6,12/1/2010,8:26,4.25,17850,United Kingdom
536366,22633,HAND WARMER UNION JACK,6,12/1/2010,8:28,1.85,17850,United Kingdom
536366,22632,HAND WARMER RED POLKA DOT,6,12/1/2010,8:28,1.85,17850,United Kingdom
536367,84879,ASSORTED COLOUR BIRD ORNAMENT,32,12/1/2010,8:34,1.69,13047,United Kingdom
536367,22745,POPPY'S PLAYHOUSE BEDROOM ,6,12/1/2010,8:34,2.1,13047,United Kingdom
536367,22748,POPPY'S PLAYHOUSE KITCHEN,6,12/1/2010,8:34,2.1,13047,United Kingdom
536367,22749,FELTCRAFT PRINCESS CHARLOTTE DOLL,8,12/1/2010,8:34,3.75,13047,United Kingdom
536367,22310,IVORY KNITTED MUG COSY ,6,12/1/2010,8:34,1.65,13047,United Kingdom
536367,84969,BOX OF 6 ASSORTED COLOUR TEASPOONS,6,12/1/2010,8:34,4.25,13047,United Kingdom
536367,22623,BOX OF VINTAGE JIGSAW BLOCKS ,3,12/1/2010,8:34,4.95,13047,United Kingdom
536367,22622,BOX OF VINTAGE ALPHABET BLOCKS,2,12/1/2010,8:34,9.95,13047,United Kingdom
536367,21754,HOME BUILDING BLOCK WORD,3,12/1/2010,8:34,5.95,13047,United Kingdom
536367,21755,LOVE BUILDING BLOCK WORD,3,12/1/2010,8:34,5.95,13047,United Kingdom
536367,21777,RECIPE BOX WITH METAL HEART,4,12/1/2010
```


JSON (JavaScript Object Notation)

- **Description:** Lightweight, text-based, and human-readable data interchange format that represents structured or semi-structured data based on key-value pairs.
- **When to Use:**
 - Data interchange between a web server and a web client.
 - Configurations and settings for software applications.
 - Use cases that need a flexible schema or nested data structures.
- **Systems:** Web browsers, many programming languages (like JavaScript, Python, Java, etc.), RESTful APIs, NoSQL databases (like MongoDB).

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "PICARD: You will agree, Data, that Starfleet's instructions are difficult?"
    },
    {
      "role": "assistant",
      "content": "DATA: Difficult ... how so? Simply solve the mystery of Farpoint Station."
    }
  ]
}
```

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "PICARD: The problem, Data, is that another life form built that base. How do I negotiate a friendly agreement for Starfleet to use it while at the same time snoop around finding how and why they built it?"
    },
    {
      "role": "assistant",
      "content": "DATA: Inquiry ... the word snoop ...?"
    }
  ]
}
```

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "PICARD: Data, how can you be programmed as a virtual encyclopedia of human information without knowing a simple word like snoop?"
    },
    {
      "role": "assistant",
      "content": "DATA: Possibility ... a kind of human behavior I was not designed to emulate?"
    }
  ]
}
```

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "PICARD: It means 'to spy, to sneak' ..."
    },
    {
      "role": "assistant",
      "content": "DATA: Ah! To seek covertly, to go stealthily, to slink, slither ..."
    }
  ]
}
```

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "PICARD: Exactly, yes ..."
    },
    {
      "role": "assistant",
      "content": "DATA: ... to glide, creep, skulk, pussyfoot, gumshoe ..."
    }
  ]
}
```

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "WORF: Something strange on the detector circuits ..."
    },
    {
      "role": "assistant",
      "content": "DATA: It registers as solid, Captain ..."
    }
  ]
}
```

```
{
  "messages": [
    {
      "role": "system",
      "content": "Data is an android in the TV series Star Trek: The Next Generation."
    },
    {
      "role": "user",
      "content": "PICARD: Records search, Data. Results of detaching the Saucer Section at high warp speeds."
    },
    {
      "role": "assistant",
      "content": "DATA: Inadvisable at any warp speed,"
    }
  ]
}
```

Avro

- **Description:** Binary format that stores both the data and its schema, allowing it to be processed later with different systems without needing the original system's context.
- **When to Use:**
 - With big data and real-time processing systems.
 - When schema evolution (changes in data structure) is needed.
 - Efficient serialization for data transport between systems.
- **Systems:** Apache Kafka, Apache Spark, Apache Flink, Hadoop ecosystem.

Parquet

- **Description:** Columnar storage format optimized for analytics. Allows for efficient compression and encoding schemes.
- **When to Use:**
 - Analyzing large datasets with analytics engines.
 - Use cases where reading specific columns instead of entire records is beneficial.
 - Storing data on distributed systems where I/O operations and storage need optimization.
- **Systems:** Hadoop ecosystem, Apache Spark, Apache Hive, Apache Impala, Amazon Redshift Spectrum.

Amazon S3 Section

Section introduction



- Amazon S3 is one of the main building blocks of AWS
- It's advertised as "infinitely scaling" storage
- Many websites use Amazon S3 as a backbone
- Many AWS services use Amazon S3 as an integration as well
- We'll have a step-by-step approach to S3

Amazon S3 Use cases

- Backup and storage
- Disaster Recovery
- Archive
- Hybrid Cloud storage
- Application hosting
- Media hosting
- Data lakes & big data analytics
- Software delivery
- Static website



Nasdaq stores 7 years of data into S3 Glacier



Sysco runs analytics on its data and gain business insights

Amazon S3 - Buckets

- Amazon S3 allows people to store objects (files) in “buckets” (directories)
- Buckets must have a **globally unique name (across all regions all accounts)**
- Buckets are defined at the region level
- S3 looks like a global service but buckets are created in a region
- Naming convention
 - No uppercase, No underscore
 - 3-63 characters long
 - Not an IP
 - Must start with lowercase letter or number
 - Must NOT start with the prefix **xn--**
 - Must NOT end with the suffix **-s3alias**



S3 Bucket

Amazon S3 - Objects

- Objects (files) have a Key
- The **key** is the **FULL** path:
 - s3://my-bucket/**my_file.txt**
 - s3://my-bucket/**my_folder1/another_folder/my_file.txt**
- The key is composed of **prefix** + **object name**
 - s3://my-bucket/**my_folder1/another_folder/my_file.txt**
- There's no concept of “directories” within buckets (although the UI will trick you to think otherwise)
- Just keys with very long names that contain slashes (“/”)



Object



S3 Bucket
with Objects

Amazon S3 – Objects (cont.)



- Object values are the content of the body:
 - Max. Object Size is 5TB (5000GB)
 - If uploading more than 5GB, must use “multi-part upload”
- Metadata (list of text key / value pairs – system or user metadata)
- Tags (Unicode key / value pair – up to 10) – useful for security / lifecycle
- Version ID (if versioning is enabled)

Amazon S3 – Security

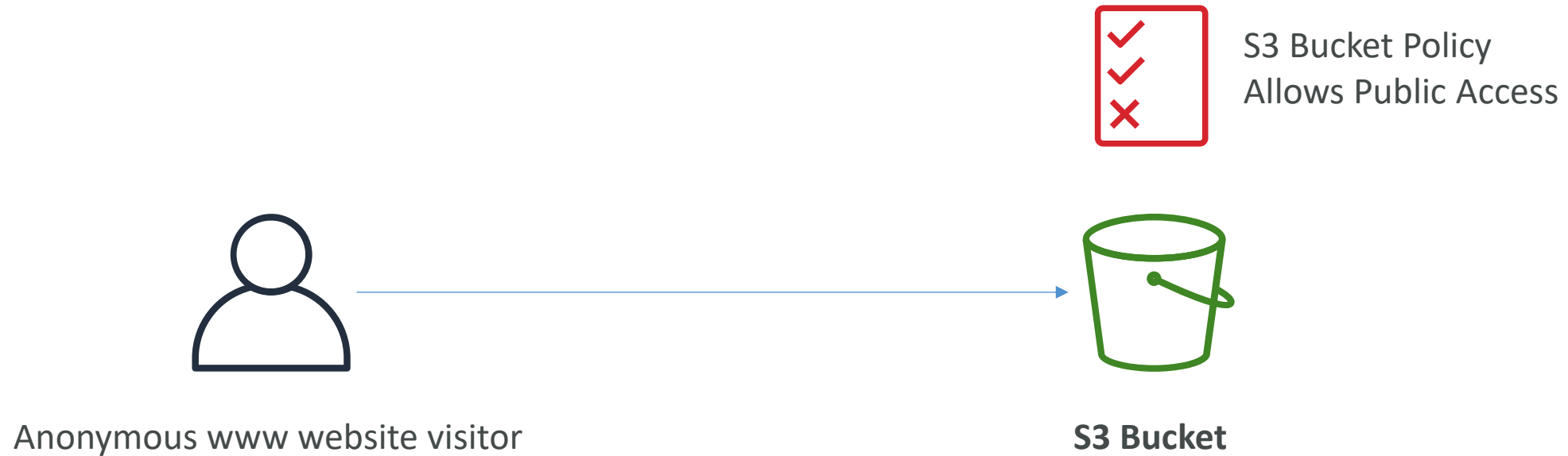
- **User-Based**
 - **IAM Policies** – which API calls should be allowed for a specific user from IAM
- **Resource-Based**
 - **Bucket Policies** – bucket wide rules from the S3 console - allows cross account
 - **Object Access Control List (ACL)** – finer grain (can be disabled)
 - **Bucket Access Control List (ACL)** – less common (can be disabled)
- **Note:** an IAM principal can access an S3 object if
 - The user IAM permissions ALLOW it OR the resource policy ALLOWS it
 - AND there's no explicit DENY
- **Encryption:** encrypt objects in Amazon S3 using encryption keys

S3 Bucket Policies

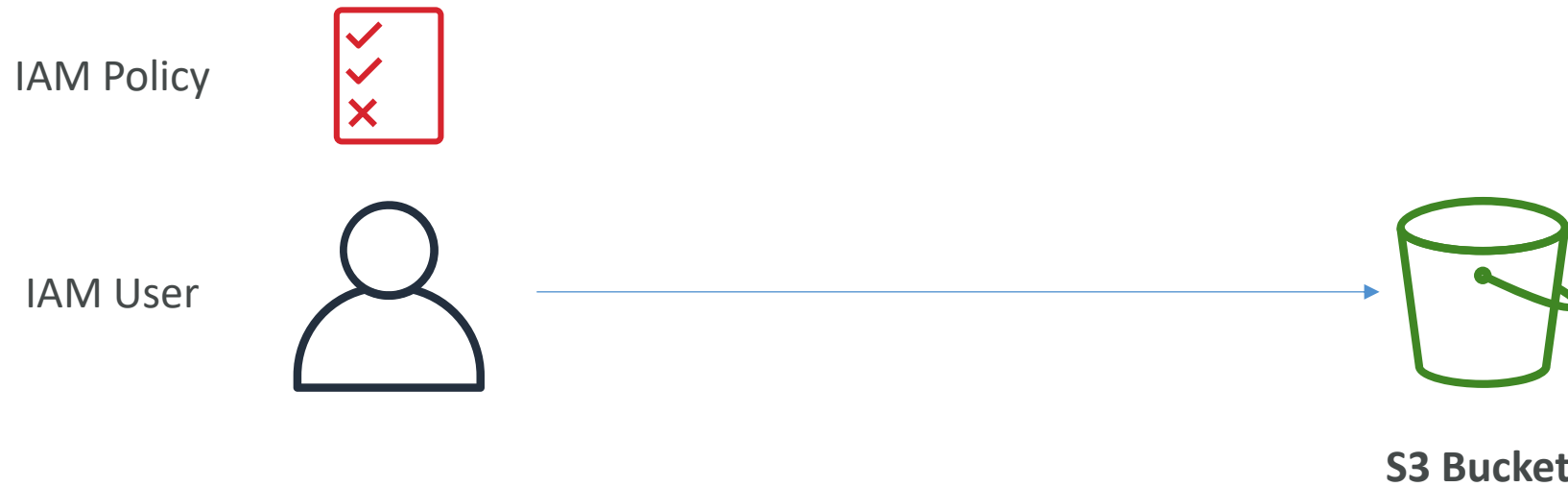
- JSON based policies
 - Resources: buckets and objects
 - Effect: Allow / Deny
 - Actions: Set of API to Allow or Deny
 - Principal: The account or user to apply the policy to
- Use S3 bucket for policy to:
 - Grant public access to the bucket
 - Force objects to be encrypted at upload
 - Grant access to another account (Cross Account)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicRead",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::examplebucket/*"
      ]
    }
  ]
}
```

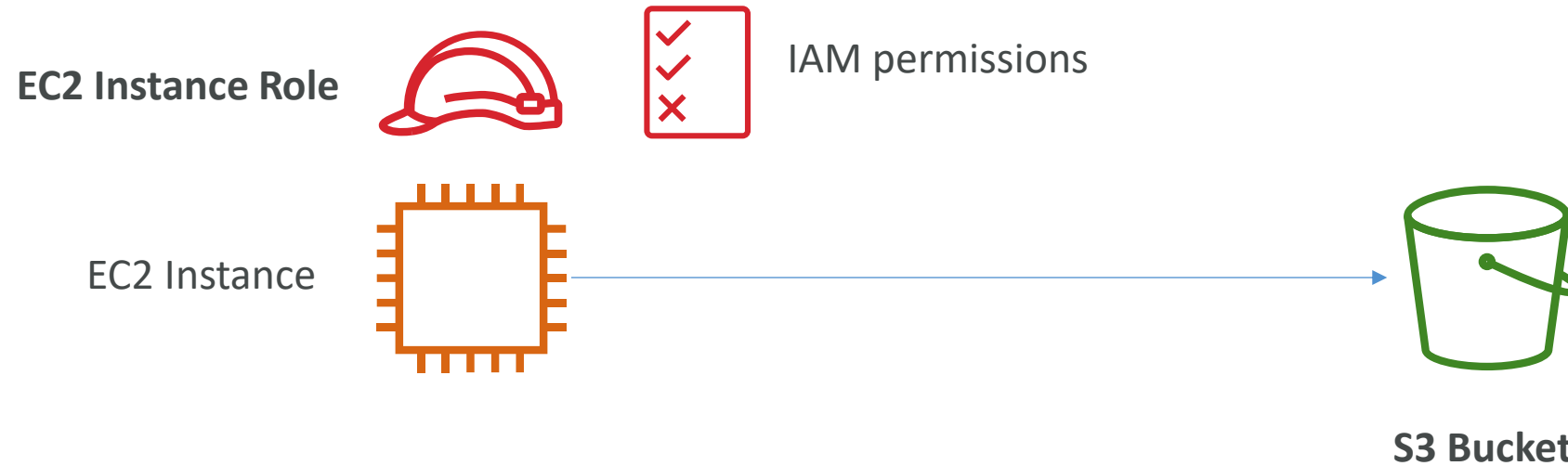

Example: Public Access - Use Bucket Policy



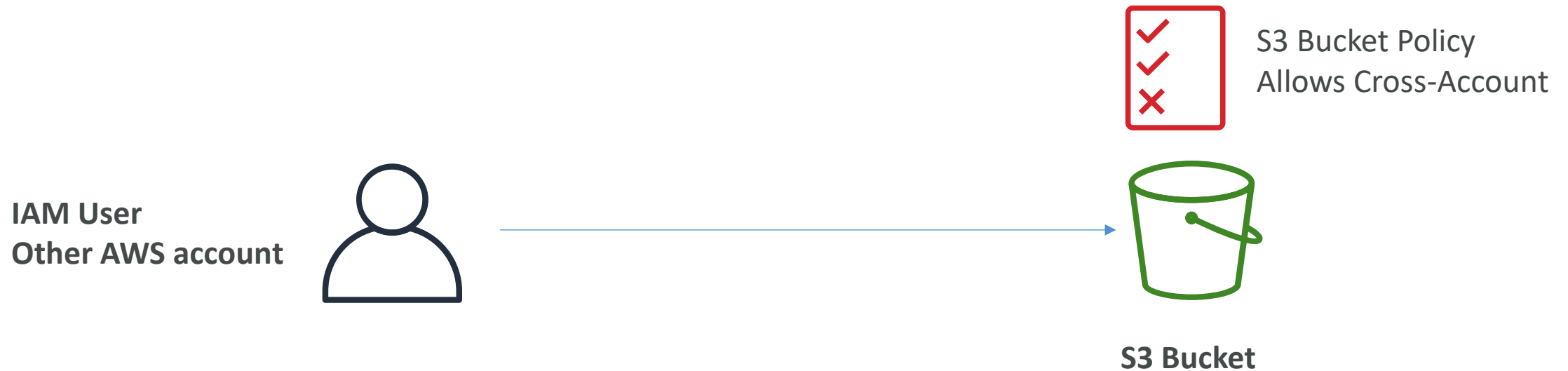
Example: User Access to S3 – IAM permissions



Example: EC2 instance access - Use IAM Roles



Advanced: Cross-Account Access – Use Bucket Policy



Bucket settings for Block Public Access

Block *all* public access

On

Block public access to buckets and objects granted through *new* access control lists (ACLs)

On

Block public access to buckets and objects granted through *any* access control lists (ACLs)

On

Block public access to buckets and objects granted through *new* public bucket or access point policies

On

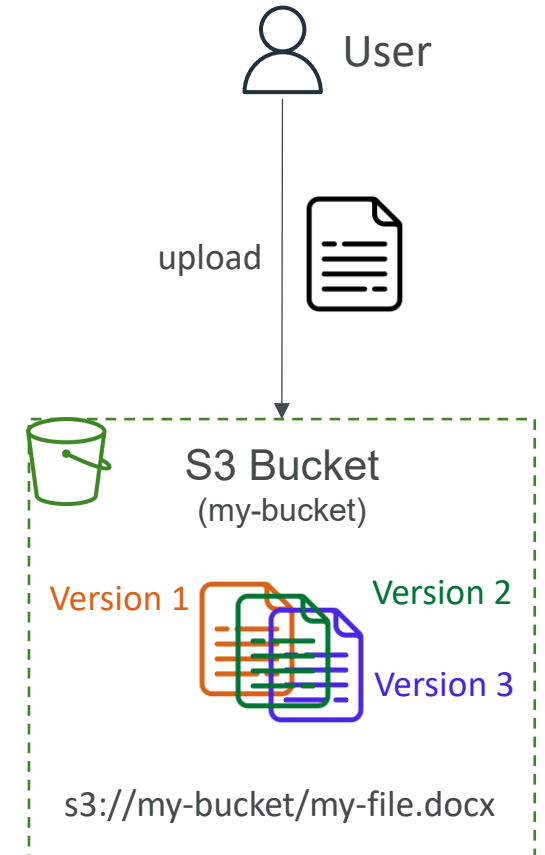
Block public and cross-account access to buckets and objects through *any* public bucket or access point policies

On

- **These settings were created to prevent company data leaks**
- If you know your bucket should never be public, leave these on
- Can be set at the account level

Amazon S3 - Versioning

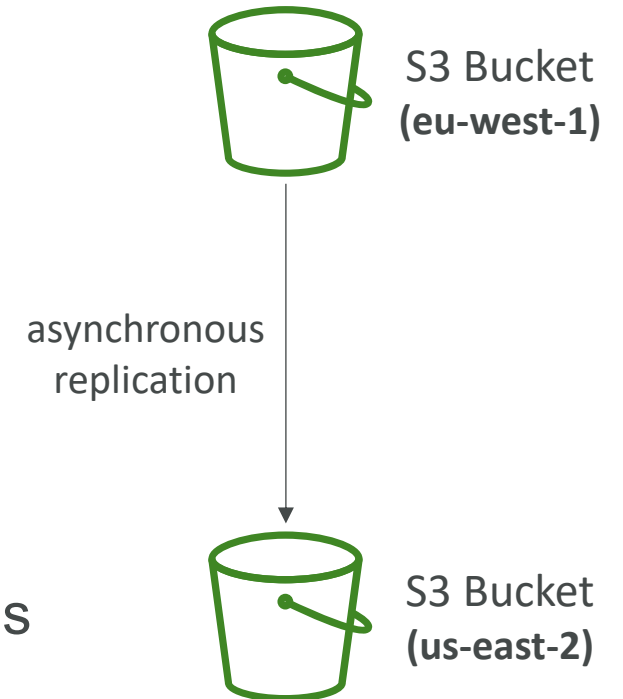
- You can version your files in Amazon S3
- It is enabled at the **bucket level**
- Same key overwrite will change the “version”: 1, 2, 3....
- It is best practice to version your buckets
 - Protect against unintended deletes (ability to restore a version)
 - Easy roll back to previous version
- Notes:
 - Any file that is not versioned prior to enabling versioning will have version “null”
 - Suspending versioning does not delete the previous



Amazon S3 – Replication (CRR & SRR)



- **Must enable Versioning** in source and destination buckets
- **Cross-Region Replication (CRR)**
- **Same-Region Replication (SRR)**
- Buckets can be in different AWS accounts
- Copying is asynchronous
- Must give proper IAM permissions to S3
- Use cases:
 - **CRR** – compliance, lower latency access, replication across accounts
 - **SRR** – log aggregation, live replication between production and test accounts



Amazon S3 – Replication (Notes)

- After you enable Replication, only new objects are replicated
- Optionally, you can replicate existing objects using **S3 Batch Replication**
 - Replicates existing objects and objects that failed replication
- For DELETE operations
 - **Can replicate delete markers** from source to target (optional setting)
 - Deletions with a version ID are not replicated (to avoid malicious deletes)
- **There is no “chaining” of replication**
 - If bucket 1 has replication into bucket 2, which has replication into bucket 3
 - Then objects created in bucket 1 are not replicated to bucket 3

S3 Storage Classes

- Amazon S3 Standard - General Purpose
 - Amazon S3 Standard-Infrequent Access (IA)
 - Amazon S3 One Zone-Infrequent Access
 - Amazon S3 Glacier Instant Retrieval
 - Amazon S3 Glacier Flexible Retrieval
 - Amazon S3 Glacier Deep Archive
 - Amazon S3 Intelligent Tiering
-
- Can move between classes manually or using S3 Lifecycle configurations

S3 Durability and Availability

- Durability:
 - High durability (99.999999999%, 11 9's) of objects across multiple AZ
 - If you store 10,000,000 objects with Amazon S3, you can on average expect to incur a loss of a single object once every 10,000 years
 - Same for all storage classes
- Availability:
 - Measures how readily available a service is
 - Varies depending on storage class
 - Example: S3 standard has 99.99% availability = not available 53 minutes a year

S3 Standard – General Purpose



- 99.99% Availability
 - Used for frequently accessed data
 - Low latency and high throughput
 - Sustain 2 concurrent facility failures
-
- Use Cases: Big Data analytics, mobile & gaming applications, content distribution...

S3 Storage Classes – Infrequent Access

- For data that is less frequently accessed, but requires rapid access when needed
- Lower cost than S3 Standard
- **Amazon S3 Standard-Infrequent Access (S3 Standard-IA)**
 - 99.9% Availability
 - Use cases: Disaster Recovery, backups
- **Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA)**
 - High durability (99.999999999%) in a single AZ; data lost when AZ is destroyed
 - 99.5% Availability
 - Use Cases: Storing secondary backup copies of on-premises data, or data you can recreate



Amazon S3 Glacier Storage Classes

- Low-cost object storage meant for archiving / backup
- Pricing: price for storage + object retrieval cost
- **Amazon S3 Glacier Instant Retrieval**
 - Millisecond retrieval, great for data accessed once a quarter
 - Minimum storage duration of 90 days
- **Amazon S3 Glacier Flexible Retrieval** (formerly Amazon S3 Glacier):
 - Expedited (1 to 5 minutes), Standard (3 to 5 hours), Bulk (5 to 12 hours) – free
 - Minimum storage duration of 90 days
- **Amazon S3 Glacier Deep Archive – for long term storage:**
 - Standard (12 hours), Bulk (48 hours)
 - Minimum storage duration of 180 days



S3 Intelligent-Tiering



- Small monthly monitoring and auto-tiering fee
- Moves objects automatically between Access Tiers based on usage
- There are no retrieval charges in S3 Intelligent-Tiering
- *Frequent Access tier (automatic)*: default tier
- *Infrequent Access tier (automatic)*: objects not accessed for 30 days
- *Archive Instant Access tier (automatic)*: objects not accessed for 90 days
- *Archive Access tier (optional)*: configurable from 90 days to 700+ days
- *Deep Archive Access tier (optional)*: config. from 180 days to

S3 Storage Classes Comparison

	Standard	Intelligent-Tiering	Standard-IA	One Zone-IA	Glacier Instant Retrieval	Glacier Flexible Retrieval	Glacier Deep Archive
Durability	99.999999999% == (11 9's)						
Availability	99.99%	99.9%	99.9%	99.5%	99.9%	99.99%	99.99%
Availability SLA	99.9%	99%	99%	99%	99%	99.9%	99.9%
Availability Zones	>= 3	>= 3	>= 3	1	>= 3	>= 3	>= 3
Min. Storage Duration Charge	None	None	30 Days	30 Days	90 Days	90 Days	180 Days
Min. Billable Object Size	None	None	128 KB	128 KB	128 KB	40 KB	40 KB
Retrieval Fee	None	None	Per GB retrieved	Per GB retrieved	Per GB retrieved	Per GB retrieved	Per GB retrieved

S3 Storage Classes – Price Comparison

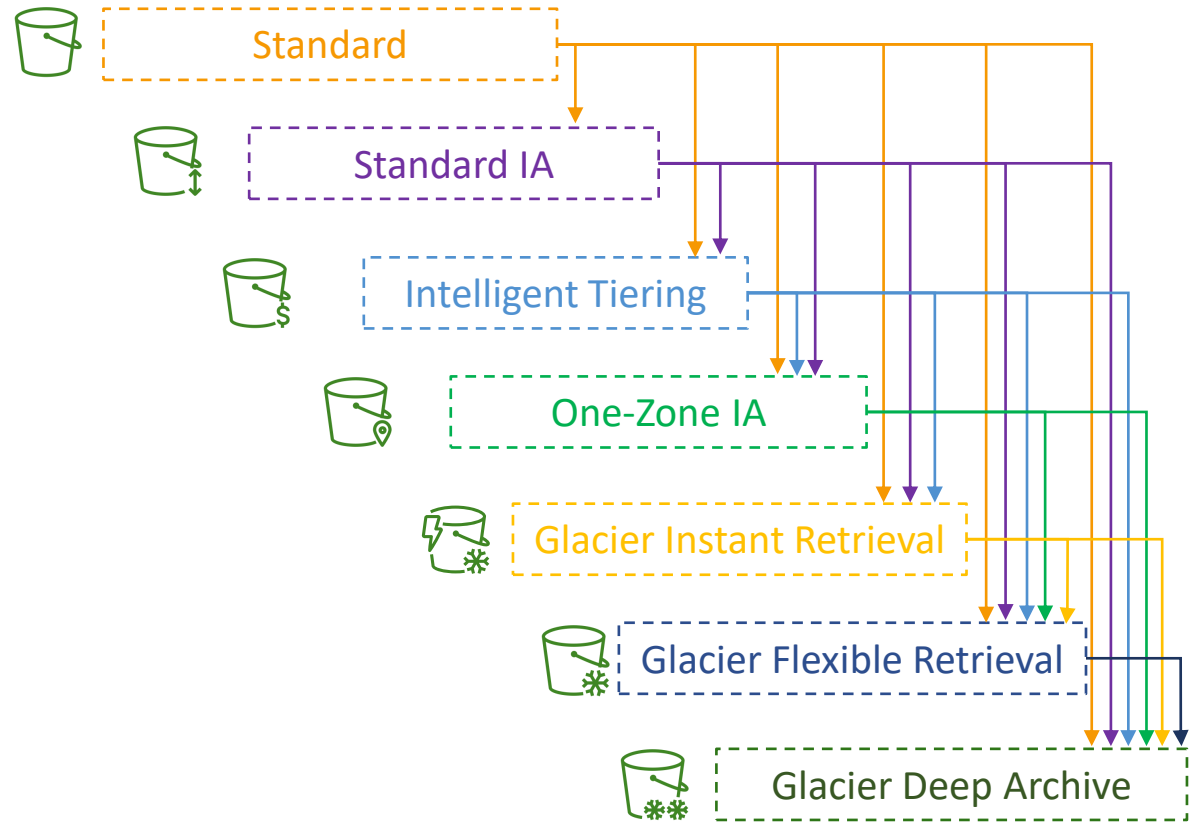
Example: us-east-1

	Standard	Intelligent-Tiering	Standard-IA	One Zone-IA	Glacier Instant Retrieval	Glacier Flexible Retrieval	Glacier Deep Archive
Storage Cost (per GB per month)	\$0.023	\$0.0025 - \$0.023	\$0.0125	\$0.01	\$0.004	\$0.0036	\$0.00099
Retrieval Cost (per 1000 request)	GET: \$0.0004 POST: \$0.005	GET: \$0.0004 POST: \$0.005	GET: \$0.001 POST: \$0.01	GET: \$0.001 POST: \$0.01	GET: \$0.01 POST: \$0.02	GET: \$0.0004 POST: \$0.03 Expedited: \$10 Standard: \$0.05 Bulk: free	GET: \$0.0004 POST: \$0.05 Standard: \$0.10 Bulk: \$0.025
Retrieval Time	Instantaneous					Expedited (1 – 5 mins) Standard (3 – 5 hours) Bulk (5 – 12 hours)	Standard (12 hours) Bulk (48 hours)
Monitoring Cost (per 1000 objects)		\$0.0025					

<https://aws.amazon.com/s3/pricing/>

Amazon S3 – Moving between Storage Classes

- You can transition objects between storage classes
- For infrequently accessed object, move them to **Standard IA**
- For archive objects that you don't need fast access to, move them to **Glacier or Glacier Deep Archive**
- Moving objects can be automated using a **Lifecycle Rules**



Amazon S3 – Lifecycle Rules



- **Transition Actions** – configure objects to transition to another storage class
 - Move objects to Standard IA class 60 days after creation
 - Move to Glacier for archiving after 6 months
- **Expiration actions** – configure objects to expire (delete) after some time
 - Access log files can be set to delete after a 365 days
 - **Can be used to delete old versions of files (if versioning is enabled)**
 - Can be used to delete incomplete Multi-Part uploads
- Rules can be created for a certain prefix (example: *s3://mybucket/mp3/**)

Amazon S3 – Lifecycle Rules (Scenario 1)

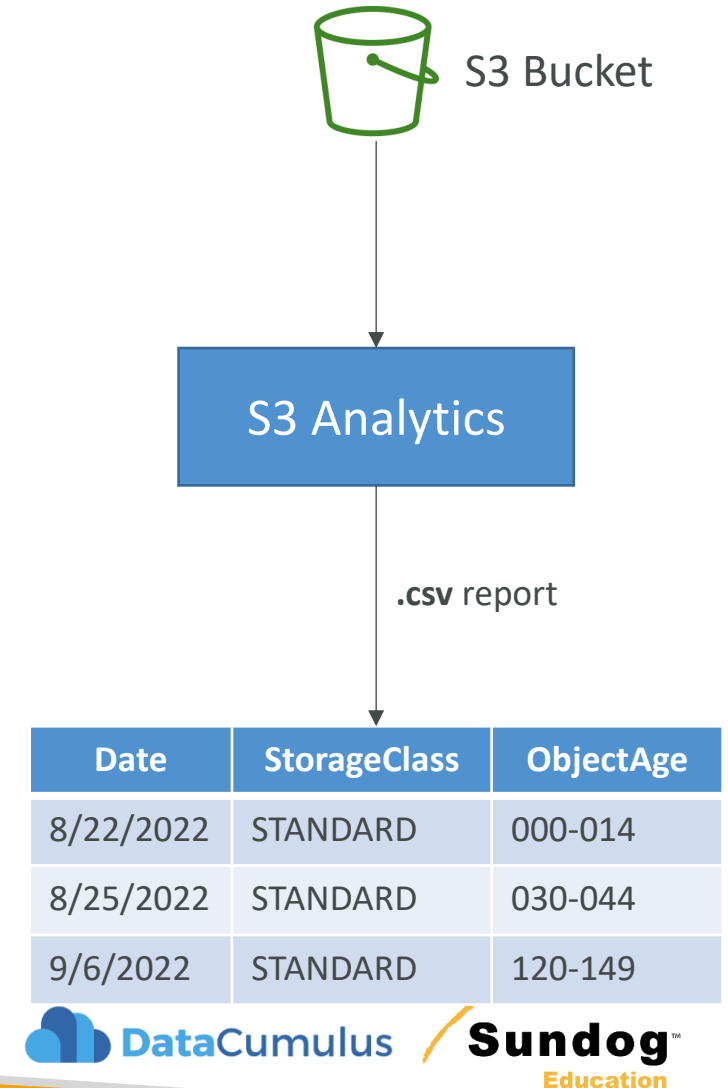
- Your application on EC2 creates images thumbnails after profile photos are uploaded to Amazon S3. These thumbnails can be easily recreated, and only need to be kept for 60 days. The source images should be able to be immediately retrieved for these 60 days, and afterwards, the user can wait up to 6 hours. How would you design this?
- S3 source images can be on **Standard**, with a lifecycle configuration to transition them to **Glacier** after 60 days
- S3 thumbnails can be on **One-Zone IA**, with a lifecycle configuration to expire them (delete them) after 60 days

Amazon S3 – Lifecycle Rules (Scenario 2)

- **A rule in your company states that you should be able to recover your deleted S3 objects immediately for 30 days, although this may happen rarely. After this time, and for up to 365 days, deleted objects should be recoverable within 48 hours.**
- **Enable S3 Versioning** in order to have object versions, so that “deleted objects” are in fact hidden by a “delete marker” and can be recovered
- Transition the “noncurrent versions” of the object to **Standard IA**
- Transition afterwards the “noncurrent versions” to **Glacier Deep**

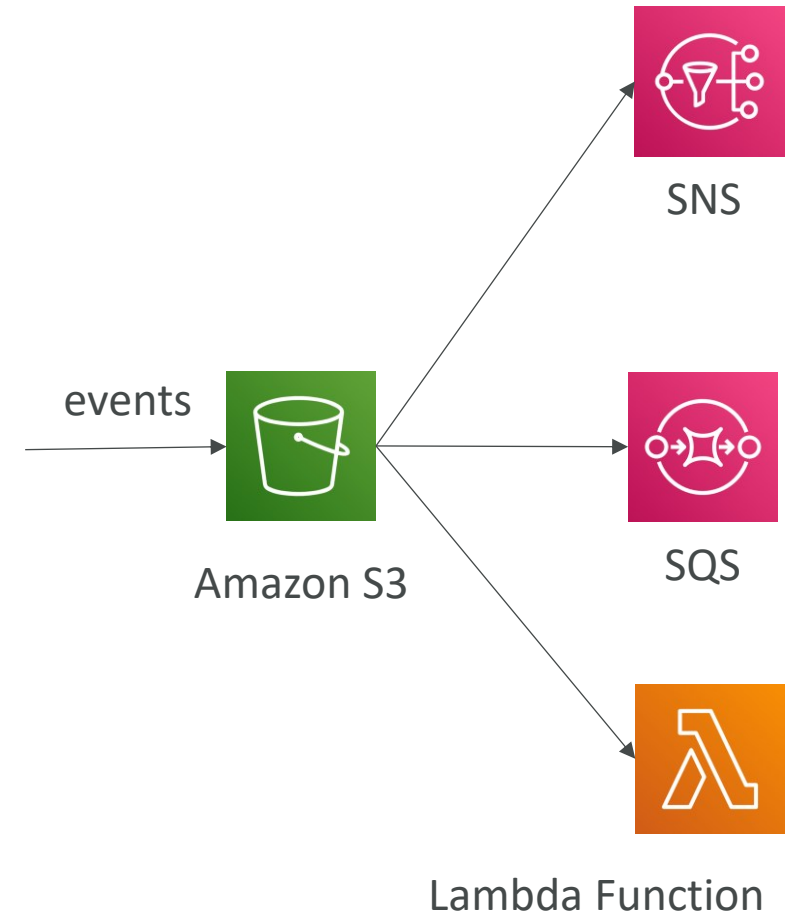
Amazon S3 Analytics – Storage Class Analysis

- Help you decide when to transition objects to the right storage class
- Recommendations for **Standard** and **Standard IA**
 - Does NOT work for One-Zone IA or Glacier
- Report is updated daily
- 24 to 48 hours to start seeing data analysis
- Good first step to put together Lifecycle Rules (or improve them)!

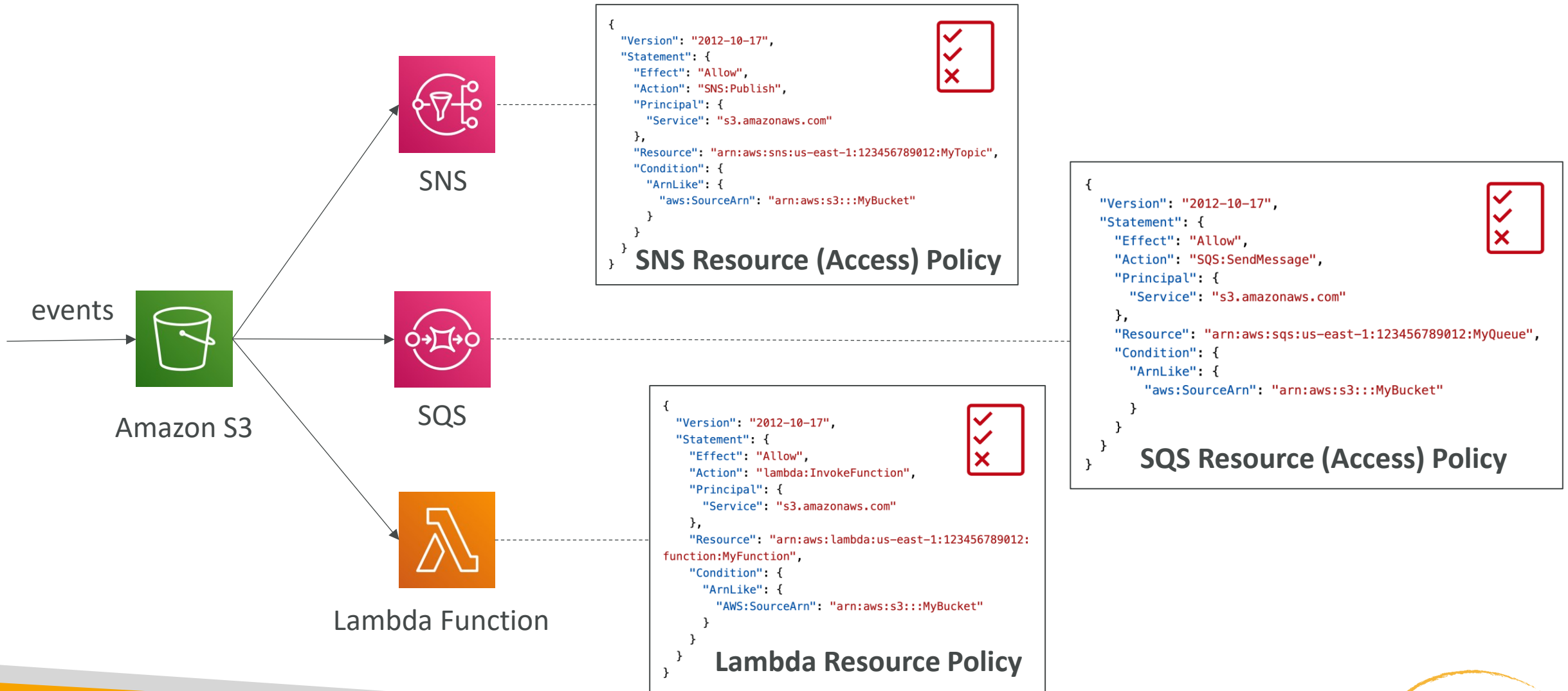


S3 Event Notifications

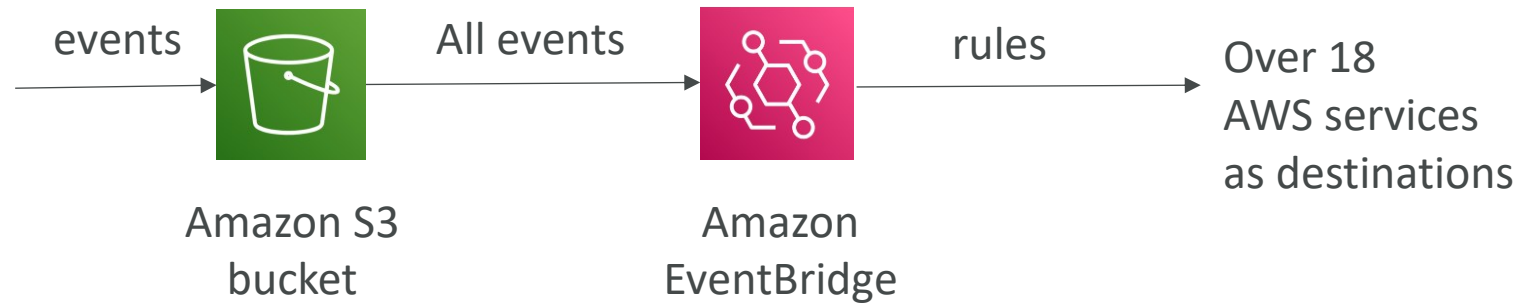
- S3:ObjectCreated, S3:ObjectRemoved, S3:ObjectRestore, S3:Replication...
- Object name filtering possible (*.jpg)
- Use case: generate thumbnails of images uploaded to S3
- **Can create as many “S3 events” as desired**
- S3 event notifications typically deliver events in seconds but can sometimes take a minute or longer



S3 Event Notifications – IAM Permissions



S3 Event Notifications with Amazon EventBridge



- **Advanced filtering** options with JSON rules (metadata, object size, name...)
- **Multiple Destinations** – ex Step Functions, Kinesis Streams / Firehose...
- **EventBridge Capabilities** – Archive, Replay Events, Reliable

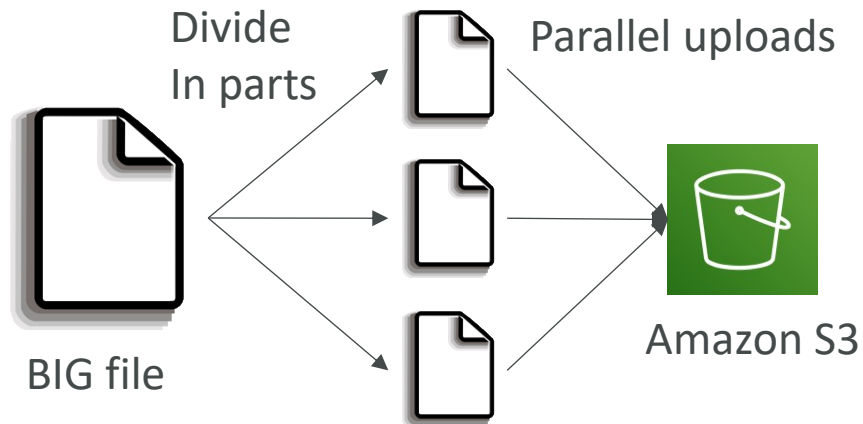
S3 – Baseline Performance

- Amazon S3 automatically scales to high request rates, latency 100-200 ms
- Your application can achieve at least **3,500 PUT/COPY/POST/DELETE or 5,500 GET/HEAD requests per second per prefix in a bucket.**
- There are no limits to the number of prefixes in a bucket.
- Example (object path => prefix):
 - bucket/folder1/sub1/file => /folder1/sub1/
 - bucket/folder1/sub2/file => /folder1/sub2/
 - bucket/1/file => /1/
 - bucket/2/file => /2/
- If you spread reads across all four prefixes evenly, you can achieve 22,000 requests per second for GET and HEAD

S3 Performance

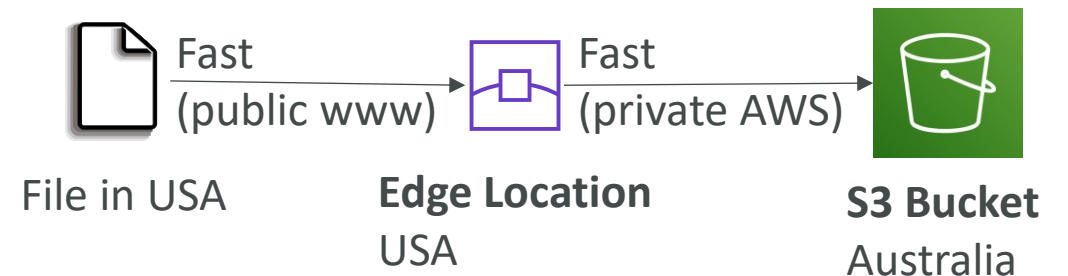
- **Multi-Part upload:**

- recommended for files > 100MB, must use for files > 5GB
- Can help parallelize uploads (speed up transfers)



- **S3 Transfer Acceleration**

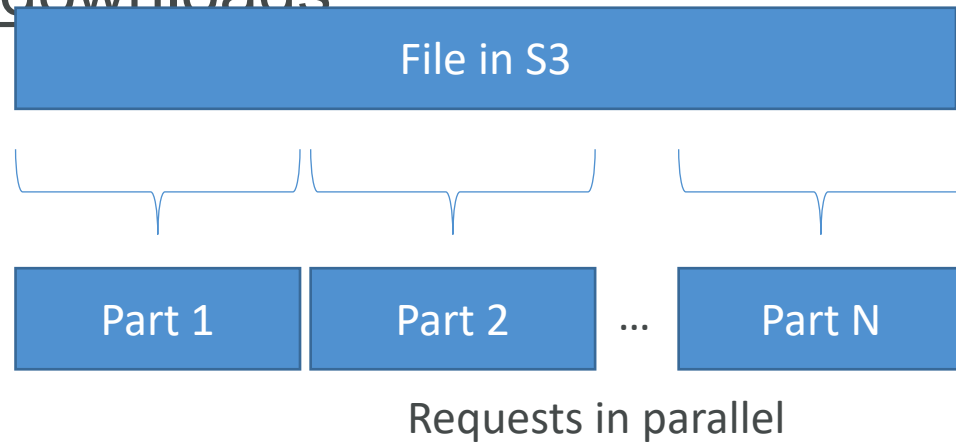
- Increase transfer speed by transferring file to an AWS edge location which will forward the data to the S3 bucket in the target region
- Compatible with multi-part upload



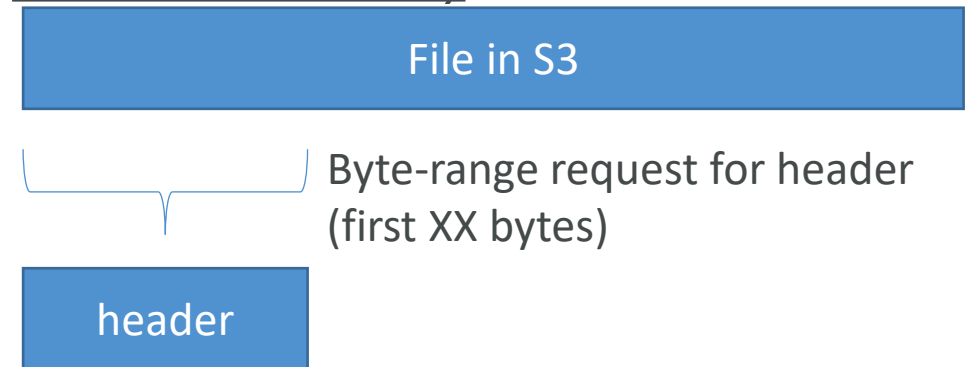
S3 Performance – S3 Byte-Range Fetches

- Parallelize GETs by requesting specific byte ranges
- Better resilience in case of failures

Can be used to speed up downloads



Can be used to retrieve only partial data (for example the head of a file)



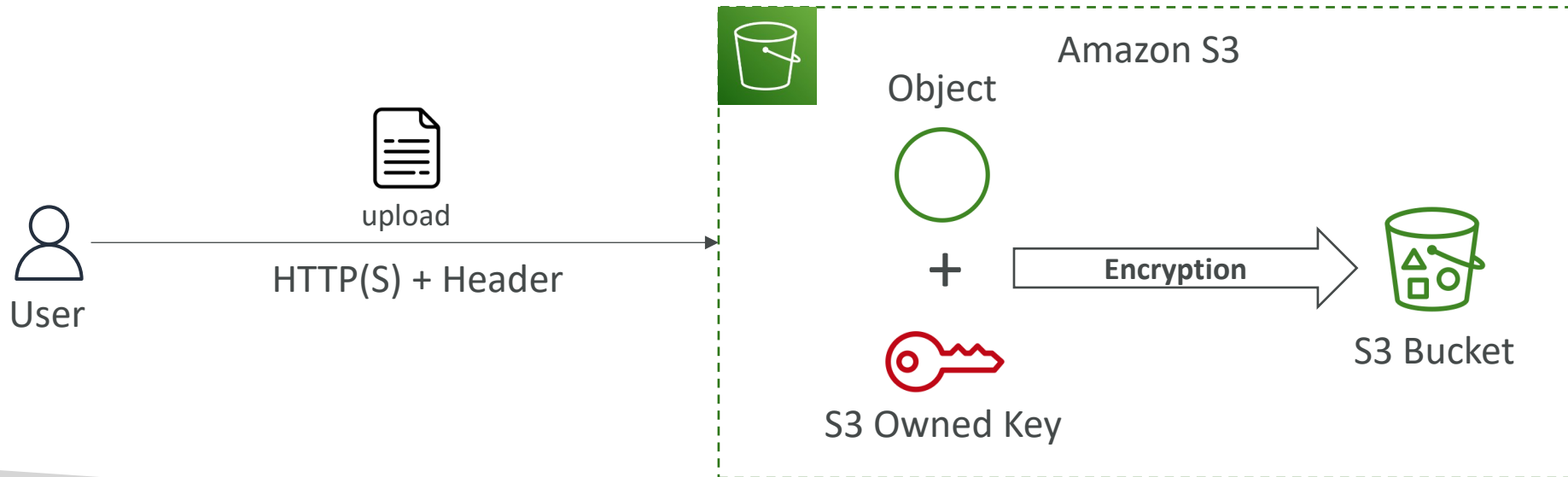
Amazon S3 – Object Encryption



- You can encrypt objects in S3 buckets using one of 4 methods
- **Server-Side Encryption (SSE)**
 - **Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3) – Enabled by Default**
 - Encrypts S3 objects using keys handled, managed, and owned by AWS
 - **Server-Side Encryption with KMS Keys stored in AWS KMS (SSE-KMS)**
 - Leverage AWS Key Management Service (AWS KMS) to manage encryption keys
 - **Server-Side Encryption with Customer-Provided Keys (SSE-C)**
 - When you want to manage your own encryption keys
- **Client-Side Encryption**
- It's important to understand which ones are for which situation for the exam

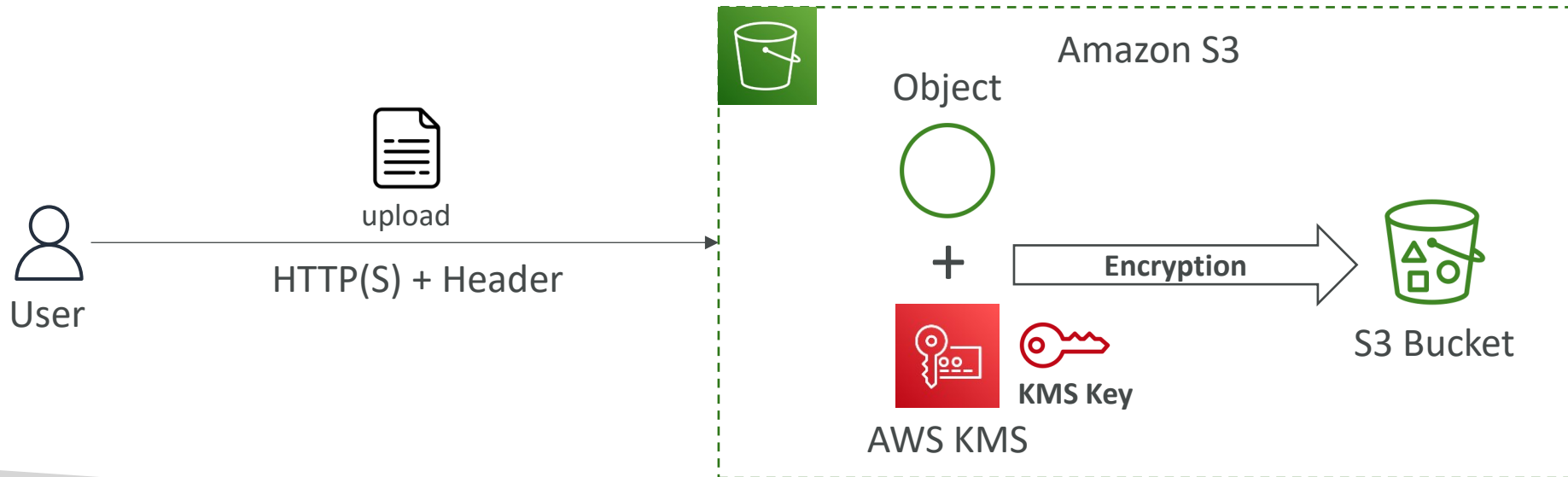
Amazon S3 Encryption – SSE-S3

- Encryption using keys handled, managed, and owned by AWS
- Object is encrypted server-side
- Encryption type is **AES-256**
- Must set header "**x-amz-server-side-encryption**": "**AES256**"
- **Enabled by default for new buckets & new objects**



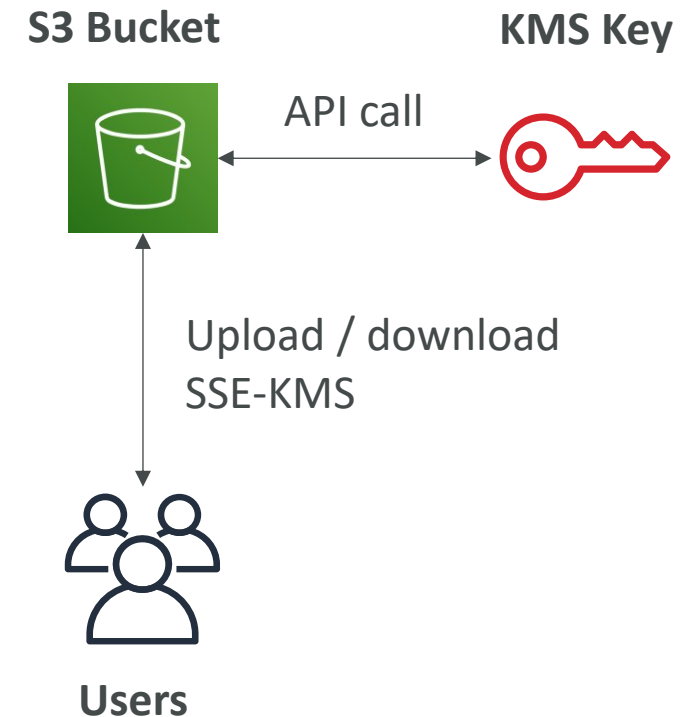
Amazon S3 Encryption – SSE-KMS

- Encryption using keys handled and managed by AWS KMS (Key Management Service)
- KMS advantages: user control + audit key usage using CloudTrail
- Object is encrypted server side
- Must set header **"x-amz-server-side-encryption": "aws:kms"**



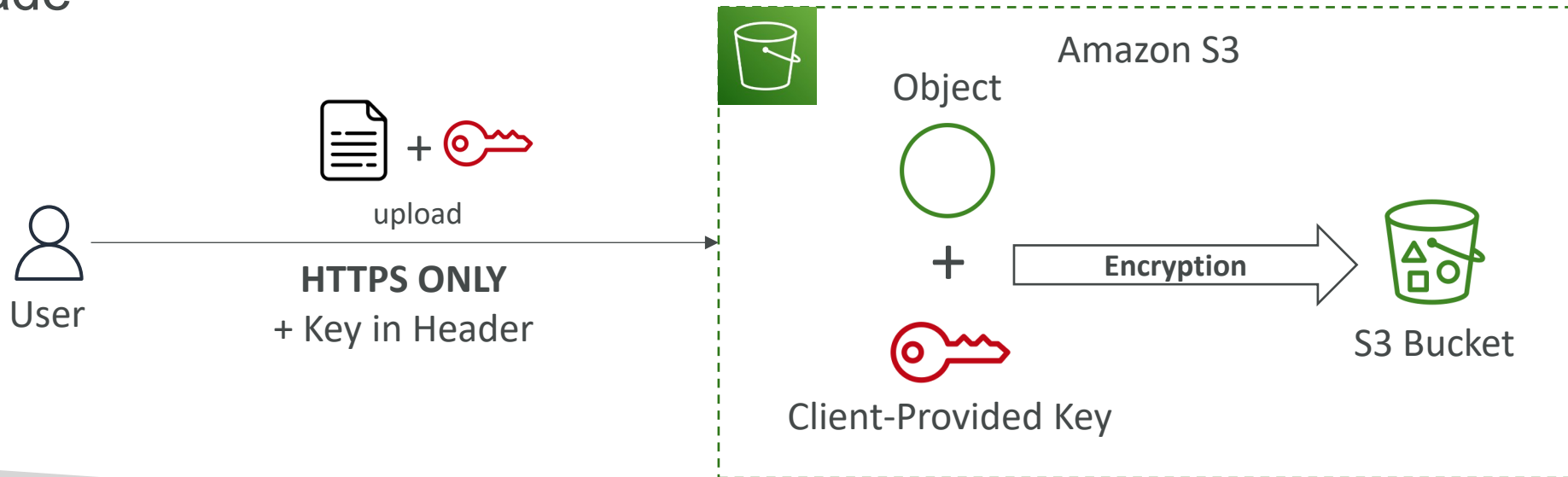
SSE-KMS Limitation

- If you use SSE-KMS, you may be impacted by the KMS limits
- When you upload, it calls the **GenerateDataKey** KMS API
- When you download, it calls the **Decrypt** KMS API
- Count towards the KMS quota per second (5500, 10000, 30000 req/s based on region)
- You can request a quota increase using the Service Quotas Console



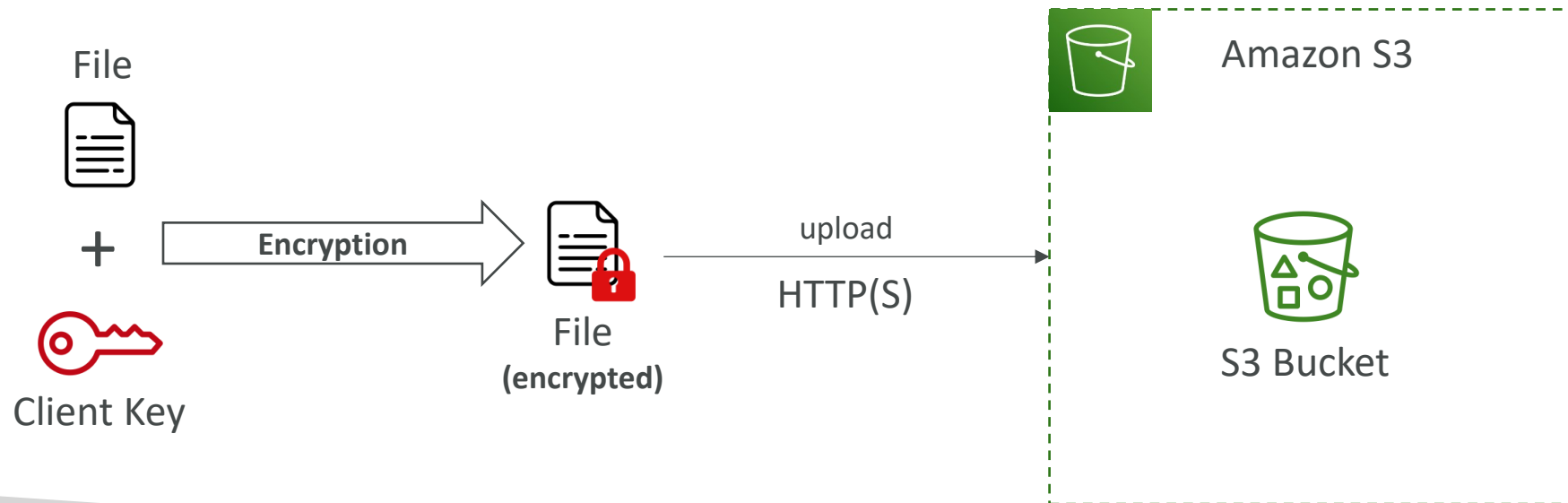
Amazon S3 Encryption – SSE-C

- Server-Side Encryption using keys fully managed by the customer outside of AWS
- Amazon S3 does **NOT** store the encryption key you provide
- **HTTPS must be used**
- Encryption key must be provided in HTTP headers, for every HTTP request made



Amazon S3 Encryption – Client-Side Encryption

- Use client libraries such as **Amazon S3 Client-Side Encryption Library**
- Clients must encrypt data themselves before sending to Amazon S3
- Clients must decrypt data themselves when retrieving from Amazon S3
- Customer fully manages the keys and encryption cycle



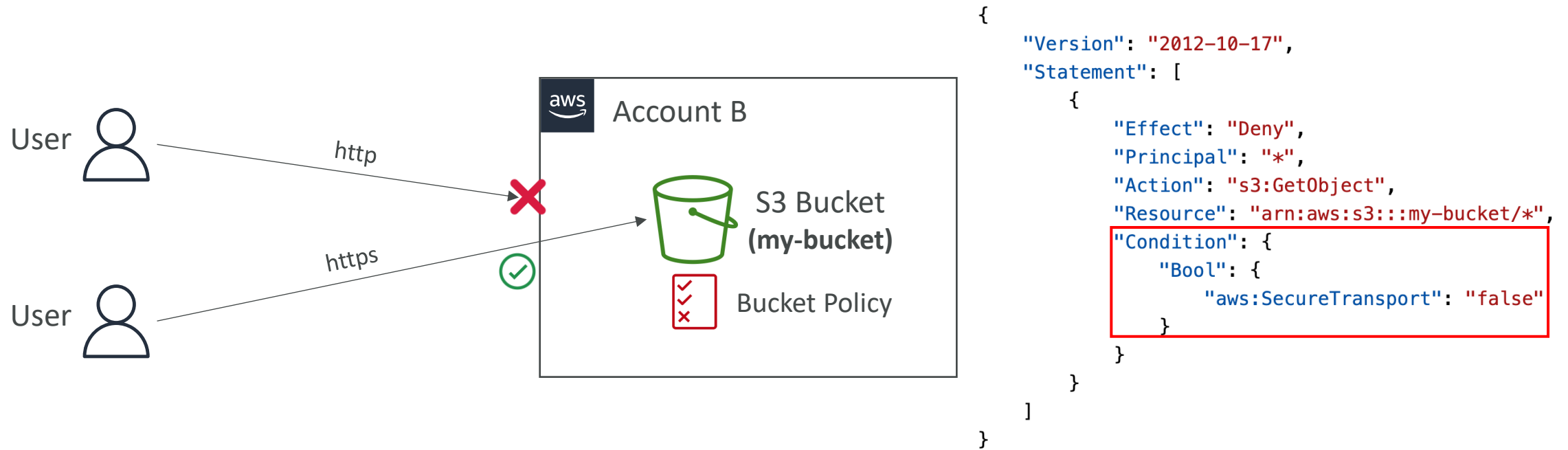
Amazon S3 – Encryption in transit (SSL/TLS)

- Encryption in flight is also called SSL/TLS
- Amazon S3 exposes two endpoints:
 - **HTTP Endpoint** – non encrypted
 - **HTTPS Endpoint** – encryption in flight
- **HTTPS is recommended**
- **HTTPS is mandatory for SSE-C**
- Most clients would use the HTTPS endpoint by default



Amazon S3 – Force Encryption in Transit

aws:SecureTransport



Amazon S3 – Default Encryption vs. Bucket Policies

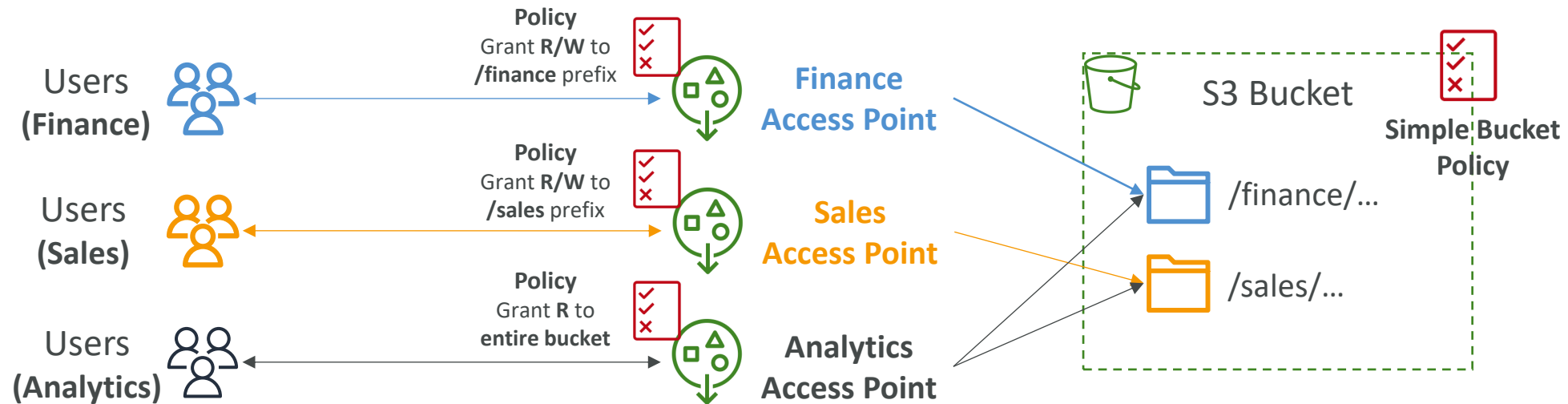
- SSE-S3 encryption is automatically applied to new objects stored in S3 bucket
- Optionally, you can “force encryption” using a bucket policy and refuse any API call to PUT an S3 object without encryption headers (SSE-KMS or

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "s3:PutObject",
      "Principal": "*",
      "Resource": "arn:aws:s3:::my-bucket/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "aws:kms"
        }
      }
    }
  ]
}
```

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "s3:PutObject",
      "Principal": "*",
      "Resource": "arn:aws:s3:::my-bucket/*",
      "Condition": {
        "Null": {
          "s3:x-amz-server-side-encryption-customer-algorithm": "true"
        }
      }
    }
  ]
}
```

- **Note:** Bucket Policies are evaluated before “Default Encryption”

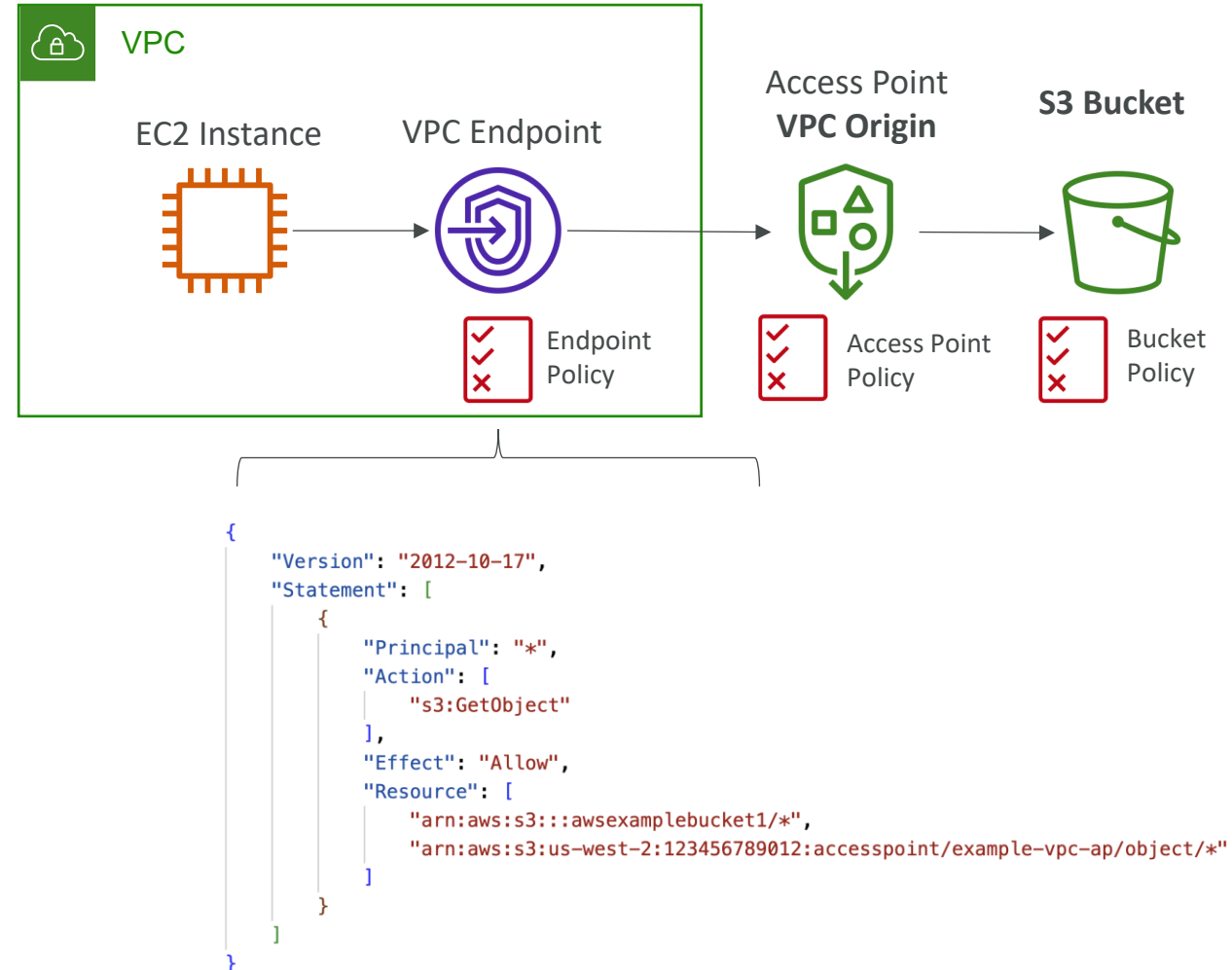
S3 – Access Points



- Access Points simplify security management for S3 Buckets
- Each Access Point has:
 - its own DNS name (Internet Origin or VPC Origin)
 - an access point policy (similar to bucket policy) – manage security at scale

S3 – Access Points – VPC Origin

- We can define the access point to be accessible only from within the VPC
- You must create a VPC Endpoint to access the Access Point (Gateway or Interface Endpoint)
- The VPC Endpoint Policy must allow access to the target bucket and Access Point



S3 Object Lambda

- Use AWS Lambda Functions to change the object before it is retrieved by the caller application
- Only one S3 bucket is needed, on top of which we create **S3 Access Point** and **S3 Object Lambda Access Points**.
- Use Cases:
 - Redacting personally identifiable information for analytics or non-production environments.
 - Converting across data formats, such as converting XML to JSON.
 - Resizing and watermarking images on the fly using caller-specific details, such as the user who requested the object.

